```
import os, json

!pip install kaggle -q
!mkdir -p ~/.kaggle

creds = {"username": "khushboopingle", "key": "YOUR_KAGGLE_KEY_HERE"}
with open('/root/.kaggle/kaggle.json', 'w') as f:
    json.dump(creds, f)
!chmod 600 ~/.kaggle/kaggle.json

# Re-download dataset if not already present
if not os.path.exists('/content/dataset'):
    !kaggle datasets download -d vipoooool/new-plant-diseases-dataset
    !unzip -q new-plant-diseases-dataset.zip -d /content/dataset
    print("✅ Dataset ready!")
else:
    print("✅ Dataset already exists!")
```

```
Dataset URL: https://www.kaggle.com/datasets/vipoooool/new-plant-diseases-dataset
License(s): copyright-authors
Downloading new-plant-diseases-dataset.zip to /content
100% 2.70G/2.70G [00:29<00:00, 99.3MB/s]

✅ Dataset ready!
```

```
!kaggle datasets download -d vipoooool/new-plant-diseases-dataset
print("Download complete!")
```

```
Dataset URL: https://www.kaggle.com/datasets/vipoooool/new-plant-diseases-dataset
License(s): copyright-authors
new-plant-diseases-dataset.zip: Skipping, found more recently modified local copy (use --force to force download)
Download complete!
```

```
!unzip -q new-plant-diseases-dataset.zip -d /content/dataset
print("Unzip complete!")
!ls /content/dataset
```

```
replace /content/dataset/New Plant Diseases Dataset(Augmented)/New Plant Diseases Dataset(Augmented)/train/Apple___Apple_scab/00
'new plant diseases dataset(augmented)'   test
'New Plant Diseases Dataset(Augmented)'
```

```
import torch
import torchvision.transforms as transforms
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader

# First let's find the correct path
import os
print(os.listdir('/content/dataset'))
```

```
['new plant diseases dataset(augmented)', 'test', 'New Plant Diseases Dataset(Augmented)']
```

```
import torch
import torchvision.transforms as transforms
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader

# Correct paths
TRAIN_DIR = '/content/dataset/New Plant Diseases Dataset(Augmented)/New Plant Diseases Dataset(Augmented)/train'
VALID_DIR = '/content/dataset/New Plant Diseases Dataset(Augmented)/New Plant Diseases Dataset(Augmented)/valid'

# Transforms
train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(30),
    transforms.ColorJitter(brightness=0.3, contrast=0.3),
    transforms.RandomResizedCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])
])

valid_transforms = transforms.Compose([
    transforms.Resize(256),
```

```
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])
])

# Load datasets
train_data = ImageFolder(TRAIN_DIR, transform=train_transforms)
valid_data = ImageFolder(VALID_DIR, transform=valid_transforms)

# Data loaders
train_loader = DataLoader(train_data, batch_size=32, shuffle=True, num_workers=2)
valid_loader = DataLoader(valid_data, batch_size=32, shuffle=False, num_workers=2)

print(f"✅ Classes found: {len(train_data.classes)}")
print(f"✅ Training images: {len(train_data)}")
print(f"✅ Validation images: {len(valid_data)}")
```

```
✅ Classes found: 38
✅ Training images: 70295
✅ Validation images: 17572
```

```
import torchvision.models as models
import torch.nn as nn

# Load pretrained ResNet-50
model = models.resnet50(weights='IMAGENET1K_V1')

# Freeze all layers
for param in model.parameters():
    param.requires_grad = False

# Unfreeze last 2 blocks for fine-tuning
for param in model.layer4.parameters():
    param.requires_grad = True
for param in model.layer3.parameters():
    param.requires_grad = True

# Replace final classifier head with 38-class head
model.fc = nn.Sequential(
    nn.Linear(2048, 512),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(512, 38)
)

# Move model to GPU if available
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)

print(f"✅ Model ready!")
print(f"✅ Using device: {device}")
print(f"✅ Trainable parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad):,}")
```

```
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to /root/.cache/torch/hub/checkpoints/resnet50-0676ba61
100%|██████████| 97.8M/97.8M [00:01<00:00, 57.7MB/s]
✅ Model ready!
✅ Using device: cuda
✅ Trainable parameters: 23,131,686
```

```
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import StepLR

# Loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam([
    {'params': model.layer3.parameters(), 'lr': 1e-4},
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(),     'lr': 1e-3}
])
scheduler = StepLR(optimizer, step_size=7, gamma=0.1)

# Training loop
EPOCHS = 10
best_acc = 0.0
```

```
for epoch in range(EPOCHS):
    model.train()
    train_loss, train_correct = 0, 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        train_loss += loss.item()
        train_correct += (outputs.argmax(1) == labels).sum().item()

    model.eval()
    val_loss, val_correct = 0, 0

    with torch.no_grad():
        for images, labels in valid_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
            val_correct += (outputs.argmax(1) == labels).sum().item()

    train_acc = 100 * train_correct / len(train_data)
    val_acc   = 100 * val_correct   / len(valid_data)

    print(f"Epoch {epoch+1}/{EPOCHS} | "
          f"Train Loss: {train_loss/len(train_loader):.3f} | "
          f"Train Acc: {train_acc:.2f}% | "
          f"Val Acc: {val_acc:.2f}%")

    if val_acc > best_acc:
        best_acc = val_acc
        torch.save(model.state_dict(), 'best_model.pth')
        print(f"  💾 Best model saved! Val Acc: {val_acc:.2f}%")

    scheduler.step()

print(f"\n🎉 Training complete! Best Val Accuracy: {best_acc:.2f}%")
```

```
Epoch 1/10 | Train Loss: 0.515 | Train Acc: 84.95% | Val Acc: 96.22%
  💾 Best model saved! Val Acc: 96.22%
Epoch 2/10 | Train Loss: 0.282 | Train Acc: 91.83% | Val Acc: 98.10%
  💾 Best model saved! Val Acc: 98.10%
Epoch 3/10 | Train Loss: 0.236 | Train Acc: 93.13% | Val Acc: 98.55%
  💾 Best model saved! Val Acc: 98.55%
Epoch 4/10 | Train Loss: 0.211 | Train Acc: 94.00% | Val Acc: 98.52%
Epoch 5/10 | Train Loss: 0.203 | Train Acc: 94.32% | Val Acc: 98.65%
  💾 Best model saved! Val Acc: 98.65%
Epoch 6/10 | Train Loss: 0.174 | Train Acc: 95.07% | Val Acc: 98.78%
  💾 Best model saved! Val Acc: 98.78%
Epoch 7/10 | Train Loss: 0.183 | Train Acc: 95.01% | Val Acc: 98.42%
Epoch 8/10 | Train Loss: 0.097 | Train Acc: 97.14% | Val Acc: 99.41%
  💾 Best model saved! Val Acc: 99.41%
Epoch 9/10 | Train Loss: 0.078 | Train Acc: 97.56% | Val Acc: 99.54%
  💾 Best model saved! Val Acc: 99.54%
Epoch 10/10 | Train Loss: 0.068 | Train Acc: 97.87% | Val Acc: 99.57%
  💾 Best model saved! Val Acc: 99.57%

🎉 Training complete! Best Val Accuracy: 99.57%
```

```
# Save model and class names to Google Drive
import json
from google.colab import drive
drive.mount('/content/drive')

torch.save(model.state_dict(), '/content/drive/MyDrive/best_model.pth')

with open('/content/drive/MyDrive/class_names.json', 'w') as f:
    json.dump(train_data.classes, f)

print("✅ Model saved to Google Drive!")
print("✅ Class names saved to Google Drive!")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
✅ Model saved to Google Drive!
```

```
✅ Class names saved to Google Drive!
```

```
Start coding or generate with AI.
```